PROJECT GUIDE

RAG-Based Intelligent Document Search and Question Answering System

How the project was planned, built, configured, and deployed

PROJECT AT A GLANCE	DETAIL
Purpose	Ask natural-language questions about PDF documents and receive answers grounded in cited pages.
Interface and hosting	Python, Streamlit, and Streamlit Community Cloud.
RAG pipeline	pypdf extraction, Sentence Transformers embeddings, NumPy similarity search, and Gemini answer generation.
Accounts and saved chats	Supabase Auth, Postgres, and row-level security policies.
Current scope	Text-based PDFs, multi-file upload, semantic retrieval, cited answers, sign-in, and per-user saved conversation history.

Prepared as a learning and presentation guide. It documents the current implementation and the practical setup steps, including the problems encountered while connecting model and hosting services.

No passwords, API keys, or database credentials are included in this guide.

Contents

- 1. Project idea and goals
- 2. Architecture and request flow
- 3. Tools and why they were chosen
- 4. Code layout
- 5. Build process
- 6. Supabase auth and privacy
- 7. Local setup and deployment
- 8. Issues encountered and resolutions
- 9. Limitations and next steps
- 10. References and attribution

1. Project idea and goals

Large PDF files can make it difficult to find a specific fact. Keyword search may miss a relevant passage when the question uses different wording. The project uses Retrieval-Augmented Generation (RAG): it retrieves document passages that are semantically related to a question, then gives those passages to a language model as evidence for an answer.

The design goal is to make the answer traceable. Each answer should use retrieved document text, name the source PDF and page, and say when the context does not contain an answer. The application also supports separate user accounts so saved conversations are private to each account.

2. Architecture and request flow

Document question-answering pipeline

STAGE	WHAT HAPPENS
1. Upload	The user selects one or more PDF files in the Streamlit interface.
2. Extract	pypdf reads selectable text one page at a time. Each extracted passage retains its source file and page number.
3. Chunk	Long page text is split into chunks of about 900 characters with 150 characters of overlap. Overlap helps preserve context across chunk boundaries.
4. Embed	Sentence Transformers converts each passage and the question into vectors locally.
5. Retrieve	NumPy computes dot products for normalized vectors, equivalent to cosine similarity, and selects the top four passages.
6. Generate	The app sends the question and retrieved passages to Gemini. The prompt instructs it to answer only from the supplied context and cite source labels.
7. Display and save	The app shows the answer and source excerpts, then saves the user message, assistant answer, and source metadata in Supabase.

Sign-in and history flow

A visitor signs up or signs in with Supabase Auth. The authenticated session identifies the user. Conversation and message rows include that user's identifier. Supabase Row Level Security (RLS) policies check the owner on every database operation, so the chat list is scoped to the signed-in account.

3. Tools and why they were chosen

TOOL	USE IN THIS APP	REASON
Python	Main application language	Common in AI and document-processing projects; supports the chosen libraries.
Streamlit	Web interface and hosting	Builds an interactive Python application without a separate front-end framework.
pypdf	PDF extraction	Extracts text by page so citations can point to page numbers.
Sentence Transformers	Semantic embeddings	Creates vectors locally so PDF text does not need to be sent to an embedding API.
NumPy	Similarity ranking	A simple dot product over normalized vectors is sufficient for this prototype's search volume.
Google GenAI SDK / Gemini	Answer generation	Produces a readable answer from retrieved context. The chosen model is configurable.
Supabase Auth	Sign-up and sign-in	Provides email/password accounts and authentication sessions.
Supabase Postgres + RLS	Persistent chat records	Stores conversations and messages and enforces per-user access in the database.
python-dotenv	Local configuration	Loads local environment variables from an ignored .env file.

4. Code layout

PATH	RESPONSIBILITY
app.py	Streamlit page, sign-in and sign-up forms, chat navigation, PDF upload, questions, and source display.
src/pdf_processor.py	Reads page text and produces overlapping chunks with page metadata.
src/rag.py	Embeds passages/questions, ranks passages, and calls the Gemini API with a grounded prompt.
src/database.py	Creates the Supabase client, restores the user's session, and reads/writes per-user conversation records.
supabase/schema.sql	Creates database tables and indexes, grants minimal authenticated access, and defines row-level security policies.
requirements.txt	Lists packages installed locally and by Streamlit Community Cloud.
.env.example	Documents local settings without containing real credentials.
.streamlit/secrets.toml.example	Documents the TOML format used for local and hosted secrets.
.gitignore	Excludes virtual environments, caches, .env, and local Streamlit secrets from Git.
README.md	Project overview, setup, deployment, limitations, and attribution.

5. Build process

Step 1: Define a small useful first version

The project began with the stated problem: finding information in large PDFs is slow and keyword search does not understand meaning. The first milestone was upload, retrieve, answer, and cite.

Step 2: Choose a reference and stack

A public MIT-licensed PDF RAG repository was used as a learning reference for the feature set. We chose a separate implementation with Streamlit, pypdf, Sentence Transformers, NumPy, Gemini, and later Supabase.

Step 3: Build the RAG path

The app extracts PDF page text, chunks it, embeds the chunks and question, ranks by similarity, and sends only the top passages to Gemini. The answer prompt asks the model to stay within the retrieved evidence.

Step 4: Add sign-in and saved history

Supabase Auth supplies accounts. Postgres holds conversation and message rows. RLS policies use auth.uid() to restrict records to the signed-in user.

Step 5: Prepare deployment

The app was deployed from GitHub to Streamlit Community Cloud. Secrets are configured in the hosting dashboard rather than checked into the repository.

Windows local setup

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
# Configure .env or .streamlit/secrets.toml
streamlit run app.py
```

Before first run, create a Supabase project and execute supabase/schema.sql in its SQL Editor. Add GEMINI_API_KEY, MODEL_NAME, SUPABASE_URL, and SUPABASE_KEY to local ignored settings. On Streamlit Cloud, enter the same values under app Settings > Secrets. The Supabase key used by the app should be the publishable key; a service-role/secret key bypasses RLS and must not be used.

6. Supabase auth and privacy

Two tables store chat data. conversations has an owner user_id, title, and creation time. messages has its parent conversation, owner user_id, role, content, source excerpt metadata, and creation time. Foreign keys cascade deletion from the user or conversation. Indexes support listing a user's chats and loading a conversation's messages.

RLS is enabled on both tables. Conversation policies only permit the authenticated owner. Message reads and inserts require the authenticated owner, and inserts also check that the parent conversation belongs to that user. These policies provide a database-level boundary even if an app query is malformed.

Email verification is enabled by default for hosted Supabase projects. A new account may need to click a confirmation link before sign-in succeeds. The app stores the conversation and messages, but not the PDF file or its embeddings; the document must be re-uploaded for a new session's questions.

7. Local setup and deployment

SETTING	EXAMPLE FORMAT	WHERE IT IS USED
GEMINI_API_KEY	A private Gemini key	Authorizes answer generation requests.
MODEL_NAME	gemini-3.8-flash	Selects the Gemini generation model.
SUPABASE_URL	https://project-ref.supabase.co	Points the client to the Supabase project API.
SUPABASE_KEY	sb_publishable_...	Identifies the application; RLS still controls row access.

Push code updates to the connected main branch. Streamlit Community Cloud rebuilds the app from GitHub. If the push is rejected because GitHub has commits absent locally, fetch and integrate the remote history before trying again; avoid force-push for this project.

8. Issues encountered and what they meant

OBSERVED ISSUE	CAUSE / INTERPRETATION	RESOLUTION
Gemini 404: old model unavailable to new users	The configured Gemini model was no longer available for this project's API access.	Updated MODEL_NAME and the code default to gemini-3.8-flash. Model availability can change; check the current Gemini model list if needed.
Gemini 503: high demand	The request reached Google, but the model service was temporarily unavailable due to demand.	Retry later. It is a remote service availability response, not a sign that the app is running locally incorrectly.
Old app remained visible after code changes	The local branch was ahead of GitHub. Streamlit deploys the GitHub branch, not unpushed local files.	Check that the updated commit is on GitHub main and wait for Cloud deployment to finish.
Push rejected: fetch first	GitHub had an additional Dev Container commit that the local branch did not contain.	Fetched and rebased the app commit on top of the remote commit, preserving both histories.
Supabase signup: Name or service not known	The host could not resolve the configured Supabase hostname, commonly due to a wrong SUPABASE_URL.	Use the project API URL from Supabase, not the dashboard page URL. Save Secrets and reboot the app.
Email not confirmed	Supabase created the account but requires the email address to be verified before sign-in.	Check inbox/spam and click the confirmation link. Configure confirmation and email delivery deliberately.

9. Limitations and next steps

- Scanned PDFs have no OCR path yet; pypdf can only extract embedded selectable text.
- PDF files and embeddings are not retained in storage. Historical messages are saved, but to continue a document conversation after a new visit, reupload and process its PDF.
- Embeddings are recreated from chunks per question. A persistent index and cached document processing would improve speed for large files.
- The system retrieves a fixed top four passages and has no reranker or automated faithfulness evaluation yet.
- The public app uses the owner's Gemini API quota. Manage usage and share access thoughtfully.
- The default hosted email sender can have delivery and rate limits; for broader sign-up, configure an appropriate SMTP provider.

Possible next development milestones

MILESTONE	LEARNING VALUE
Add OCR for image-only pages	Understand OCR and page-level fallback handling.
Persist document metadata and embeddings	Learn storage, indexing, and document ownership.
Add chat deletion and export	Practice safe lifecycle management for user data.
Create a small evaluation question set	Measure retrieval relevance, citation correctness, and refusal when context is missing.
Add retry/backoff for temporary API errors	Improve behavior during rate limits and temporary service outages.

10. References and attribution

This project was developed as a learning implementation with the following documentation and reference project:

- Reference PDF RAG project (MIT): https://github.com/ki11e6/pdf-chatbot-rag
- Streamlit Community Cloud deployment: https://docs.streamlit.io/deploy/streamlit-community-cloud/deploy-your-app/deploy
- Streamlit secrets: https://docs.streamlit.io/deploy/streamlit-community-cloud/deploy-your-app/secrets-management
- Supabase Auth: https://supabase.com/docs/guides/auth
- Supabase Row Level Security: https://supabase.com/docs/guides/database/postgres/row-level-security
- Google Gemini models: https://ai.google.dev/gemini-api/docs/models

The reference repository is MIT-licensed. If code is copied from it, preserve the required copyright and license notice. This guide describes the separate implementation in this project workspace.